코드 구조 분석

개요

libiop 라이브러리는 현대적인 C++ 설계 원칙을 따라 구현된 zkSNARK 프로토콜 라이브러리입니다. 템플릿 기반 설계를 통해 다양한 유한체 타입을 지원하며, 계층적 클래스 구조를 통해 코드의 재사용성과 확장성을 극대화했습니다.

템플릿 기반 설계와 필드 타입 추상화

필드 타입 추상화

libiop의 핵심 설계 철학은 **필드 타입 추상화**입니다. 모든 주요 클래스와 함수는 `FieldT` 템플릿 매개변수를 사용하여 다양한 유한체에서 동작할 수 있도록 설계되었습니다.

```
template<typename FieldT>

class iop_protocol {

// 모든 핵심 기능이 FieldT에 대해 일반화됨

std::vector<field_subset<FieldT>> domains_;

std::vector<oracle<FieldT>> oracles_;

std::vector<std::vector<FieldT>> prover_messages_;

};
```

이러한 설계는 다음과 같은 장점을 제공합니다:

- **타입 안전성**: 컴파일 타임에 타입 검사가 이루어져 런타임 오류를 방지

- **성능 최적화**: 각 필드 타입에 특화된 최적화된 코드 생성
- **확장성**: 새로운 필드 타입 추가 시 기존 코드 수정 없이 지원 가능

템플릿 특수화 패턴

라이브러리는 필드 타입의 특성에 따른 템플릿 특수화를 활용합니다:

```
// 곱셈 그룹 기반 필드를 위한 특수화
template<typename FieldT>
void field_subset<FieldT>::construct_internal(
const std::size_t  num_elements,
const  typename libff::enable_if<libff::is_multiplicative<FieldT>::value, FieldT


// 덧셈 그룹 기반 필드를 위한 특수화
template<typename FieldT>
void field_subset<FieldT>::construct_internal(
const std::size_t  num_elements,
const  typename libff::enable_if<libff::is_additive<FieldT>::value, FieldT>::typ
```

클래스 계층 구조와 상속 관계

핸들 기반 아키텍처

libiop는 **핸들 기반 아키텍처**를 사용하여 다양한 IOP 구성 요소를 관리합니다:

```
enum iop_handle_type {
```

```
domain_handle_type = 1,
prover_message_type = 2,
verifier_random_message_type = 3,
query_type = 4,
};


template<iop_handle_type type>
class handle {
protected:
std::size_t id_;
public:
explicit  handle(const std::size_t  id) : id_(id) {}
std::size_t  id() const { return  this->id_; }
};
```

이 설계는 다음과 같은 계층 구조를 형성합니다:

```
handle<T> (기본 핸들 클래스)
├── domain_handle (도메인 핸들)
├── prover_message_handle (증명자 메시지 핸들)
├── verifier_random_message_handle (검증자 랜덤 메시지 핸들)
└── query_handle (쿼리 핸들)
```

오라클 계층 구조

오라클 시스템은 다음과 같은 상속 관계를 가집니다:

```
oracle_handle_base (기본 오라클 핸들)
├── oracle_handle (일반 오라클)
└── virtual_oracle_handle (가상 오라클)
```

가상 오라클은 여러 기본 오라클의 조합으로 구성되며, 지연 평가(lazy evaluation)를 지원합니다:

```
class virtual_oracle_registration {
domain_handle domain_;
std::size_t degree_;
std::vector<oracle_handle_ptr> constituent_oracles_;
};
```

프로토콜 계층 구조

각 zkSNARK 프로토콜은 공통 인터페이스를 구현하면서도 고유한 특성을 가집니다:

```
iop_protocol<FieldT> (기본 IOP 프로토콜)
└── bcs_protocol<FieldT, MT_hash_type> (BCS 변환 프로토콜)
├── ligero_iop<FieldT> (Ligero 프로토콜)
├── aurora_iop<FieldT> (Aurora 프로토콜)
└── fractal_iop<FieldT> (Fractal 프로토콜)
```

네임스페이스 구성과 모듈 간 의존성

네임스페이스 구조

libiop는 단일 네임스페이스 `libiop` 를 사용하여 모든 구성 요소를 포함합니다. 이는 다음과 같은 장점을 제공합니다:

- **단순성**: 복잡한 네임스페이스 계층 없이 명확한 구조
- **일관성**: 모든 구성 요소가 동일한 네임스페이스에서 접근 가능
- **상호 운용성**: 모듈 간 쉬운 상호 작용

모듈 구조

라이브러리는 다음과 같은 주요 모듈로 구성됩니다:

```
libiop/
├── algebra/ # 수학적 기초 (유한체, 다항식, FFT)
| ├── field_subset/ # 필드 부분집합 (부공간, 코셋)
| └── polynomials/ # 다항식 연산
├── bcs/ # BCS 변환 구현
| └── hashing/ # 해시 함수 추상화
├── common/ # 공통 유틸리티
├── iop/ # IOP 기본 인터페이스
├── protocols/ # 구체적인 프로토콜 구현
| ├── encoded/ # 인코딩된 프로토콜
| └── ldt/ # 저차 테스트
├── relations/ # R1CS 관계 시스템
```

```
└── snark/ # 완전한 SNARK 구현
```

의존성 관계

모듈 간 의존성은 다음과 같은 계층적 구조를 따릅니다:

```
snark (최상위 레벨)
↓
protocols (프로토콜 구현)
↓
iop + bcs (IOP 인터페이스 + BCS 변환)
↓
relations + algebra (관계 시스템 + 수학적 기초)
↓
common (공통 유틸리티)
```

핵심 의존성 패턴

1. **상향 의존성**: 하위 모듈은 상위 모듈을 참조하지 않음
2. **인터페이스 분리**: 각 모듈은 명확한 인터페이스를 통해 상호작용
3. **템플릿 기반 결합**: 컴파일 타임에 의존성이 해결됨

순환 의존성 방지

라이브러리는 다음과 같은 방법으로 순환 의존성을 방지합니다:

- **전방 선언**: 헤더 파일에서 클래스 전방 선언 사용

- **구현 분리**: 템플릿 구현을 `.tcc` 파일로 분리
- **인터페이스 추상화**: 구체적인 구현 대신 추상 인터페이스 의존

설계 패턴과 아키텍처 원칙

RAII (Resource Acquisition Is Initialization)

모든 리소스 관리는 RAII 패턴을 따릅니다:

```
template<typename FieldT, typename MT_hash_type>
class merkle_tree {
// 생성자에서 리소스 할당
merkle_tree(const std::size_t  num_leaves, ...);
// 소멸자에서 자동 정리 (컴파일러 생성)
~merkle_tree() = default;
};
```

팩토리 패턴

복잡한 객체 생성은 팩토리 메서드를 통해 처리됩니다:

```
domain_handle register_subspace(const affine_subspace<FieldT> &S);
domain_handle register_coset(const multiplicative_coset<FieldT> &S);
domain_handle register_domain(const field_subset<FieldT> &S);
```

전략 패턴

해시 함수와 같은 알고리즘은 전략 패턴으로 추상화됩니다:

```
template<typename FieldT, typename hash_digest_type>

class merkle_tree {

std::shared_ptr<leafhash<FieldT, hash_digest_type>> leaf_hasher_;

two_to_one_hash_function<hash_digest_type> node_hasher_;

};
```

이러한 구조적 설계는 libiop가 복잡한 암호학적 프로토콜을 효율적이고 안전하게 구현할 수 있는 기반을 제공합니다.